"REAL-TIME SIGN LANGUAGE INTERPRETER"

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINNERING

By

M Sri Nithya	2520030447
Ananya Roy	**2520030471**
D Shreenitha	**2520030504**

Under the Esteemed Guidance of

Dr. Raghvendra
Assistant Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOL



Declaration

The Project Report entitled **Real-Time Sign Language Interpreter** is a record of Bonafide work of M Sri Nithya 2520030447, **Ananya Roy** 2520030471, D Shreenitha2520030504 submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

M SRI NITHYA	**2520030447**
ANANYA ROY	**2520030471**
D SHREENITHA	**2520030504**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled **"Real-Time Sign Language Interpreter"** is a bonafide work done and submitted by M Sri Nithya 2520030447, **Ananya Roy**2520030471, D Shreenitha2520030504,in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2024-2025.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr. Raghvendra**, and my guide **Dr. Raghvendra,** Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)**, Head of the ***Department for Computer Science Engineering*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

M SRI NITHYA 2520030447

ANANYA ROY 2520030471

D SHREENITHA 2520030504

Abstract

The Real-Time Sign Language Interpreter (SignVision) is a machine learning-based system designed to recognize hand gestures representing sign language alphabets and convert them into readable text. The main objective of this project is to reduce communication barriers between sign language users and people who are unfamiliar with sign language.

The system uses a labeled ASL alphabet image dataset. The provided training script loads up to 300 images per class from class-named folders. Logistic Regression and Random Forest use grayscale images resized to 32×32 pixels, normalized to 0–1, and flattened into 1,024 features. The CNN uses RGB images resized to 64×64 pixels and normalized to 0–1. The training script compares Logistic Regression, Random Forest, and CNN using accuracy, weighted precision, weighted recall, weighted F1-score, classification reports, and a CNN confusion matrix. The webcam script loads the saved Random Forest model and label encoder, predicts gestures from a central camera region, smooths predictions across recent frames, and lets users build a text string from recognized letters.

The trained model can be integrated into a simple application that accepts uploaded images or webcam input to predict the corresponding sign language alphabet. This project demonstrates the machine learning lifecycle, from data preprocessing and model training to evaluation and deployment. SignVision aims to explore how artificial intelligence can support accessible communication and contribute to the development of assistive technologies.

Keywords: Sign language recognition, machine learning, computer vision, image classification, CNN, Logistic Regression, Random Forest, accessibility

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		1–5
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		6–12
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		13–17
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		18–22
		4.1 C / C++ Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 POSIX Threads / Synchronization	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	
5	System Architecture and Design		23–29
		5.1 Overall System Architecture	
		5.2System Workflow	
		5.3 Flowchart / Data Flow	

		5.4 Module Description	
		5.5 Input and Output Design	
6	**Implementation**		**30–45**
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C / C++ Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process / Thread Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 Memory Management Implementation	
		6.8 Optional User Interface Implementation	
7	**Execution and Demonstration**		**62–67**
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	**Conclusion**		**79–83**
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	**References**		**84–86**
		9.1 Books	
		9.2 Research Papers	
		9.3 Linux / GNU Documentation	
		9.4 Online Resources	

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Sign language is used by many people who are deaf or hard of hearing and by others who communicate through visual gestures. Hand shape, finger position, orientation, and movement can carry meaning. People who do not know a sign language may find it difficult to understand these gestures, which can create barriers in everyday communication, education, and public services.

Recent advances in machine learning and computer vision make it possible to train models to recognize visual patterns in images. SignVision explores this capability by focusing on static hand gestures that represent sign language alphabets. A trained classifier can take an image of a hand gesture and predict the corresponding alphabet class, which can then be displayed as text.

1.2 Problem Statement

Manual interpretation of sign language may not always be available, and people unfamiliar with sign language can struggle to understand alphabetic hand gestures. The project addresses the problem of automatically classifying sign language alphabet images and presenting the predicted character in a simple, understandable form. The initial scope is alphabet recognition; it does not claim full translation of continuous sign language or sentence-level meaning.

1.3 Scope of the Project

Use a labeled image dataset containing sign language alphabet gestures.

Prepare images through resizing, normalization, and other appropriate preprocessing.

Apply data augmentation when suitable to improve variation in training samples.

Compare Logistic Regression, Random Forest, and CNN-based image classification approaches.

Evaluate classifiers using accuracy, precision, recall, F1-score, and a confusion matrix.

Provide a basic interface for image upload and, if implemented, webcam-based predictions.

Display the predicted alphabet as readable text and document the limitations of the model.

1.4 Objectives of the Project

Understand and prepare a labeled dataset for sign language alphabet recognition.

Build a repeatable machine learning pipeline for training and testing.

Compare selected traditional machine learning models with a CNN approach.

Measure classification performance with multiple evaluation metrics.

Develop a simple way to submit an image or capture a gesture and view the predicted class.

Explore how computer vision can contribute to accessible communication tools.

1.5 Importance of the Application

SignVision is an educational prototype that demonstrates a possible assistive application of machine learning. It can help learners explore image classification and can serve as a foundation for future accessibility tools. Its predictions should be treated as model outputs rather than a replacement for qualified human interpretation, particularly for real-world communication.

1.6 Target Users / Audience

Students learning machine learning and computer vision.

Developers exploring assistive technology prototypes.

Educators demonstrating image classification.

People interested in sign language alphabet recognition and accessibility research.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

- Computer or laptop capable of running Python and machine learning libraries.
- Recommended: 8 GB RAM or more for comfortable experimentation; smaller datasets may run with less memory.
- Storage space for the dataset, source code, trained model, and outputs
- Webcam only if live camera input is implemented.
- GPU is optional; a compatible GPU may accelerate CNN training but is not mandatory for a small prototype.

2.2 Software Requirements

- Operating system: Windows, Linux, or macOS; a cloud notebook may also be used.
- Python 3.x.
- Jupyter Notebook, Google Colab, or Visual Studio Code.
- Libraries as required by the implementation: NumPy, pandas, Matplotlib, scikit-learn, and TensorFlow/Keras or PyTorch.
- OpenCV or a similar image library if image capture or webcam processing is used.

2.3 Development Environment

The project can be developed locally or in Google Colab. The selected environment should support the installed Python version and libraries. If a webcam is used, permissions and camera availability must be checked. Exact versions should be recorded from the environment actually used.

2.4 Dataset Requirements

The dataset should contain labeled images for the alphabet classes included in the project. Before training, inspect class names, image counts, image dimensions, corrupted files, duplicates, and class imbalance. The dataset source and license should be cited in the final report. The dataset name and download URL should be added after the team confirms the actual dataset used.

CHAPTER 3 - BACKGROUND STUDY AND RELATED APPROACHES

3.1 Logistic Regression

Logistic Regression is a supervised classification algorithm commonly used as a baseline. For image tasks, images generally need to be resized and represented as numerical feature vectors before training. It is relatively simple to train and interpret, but flattened pixel data may not represent spatial patterns as effectively as a CNN.

3.2 Random Forest

Random Forest combines predictions from multiple decision trees. It can model non-linear relationships and is often useful as a baseline when images are represented as feature vectors. However, training directly on high-dimensional raw pixels can be computationally demanding, and the method does not inherently learn spatial image features in the same way as convolutional networks.

3.3 Convolutional Neural Network (CNN)

A CNN is a deep learning architecture designed for grid-like data such as images. Convolutional layers learn visual patterns, pooling layers can reduce spatial dimensions, and dense layers produce class scores. A CNN can learn features such as edges, contours, and hand shapes from training images, but its performance depends on data quality, model design, training settings, and sufficient examples.

3.4 Proposed Approach

The proposed workflow is to load and inspect the labeled dataset, preprocess images, split the data into training and testing sets, train the selected models, compare their metrics, and choose a suitable model for integration. All preprocessing parameters and class labels should be applied consistently at inference time. The final interface may accept an uploaded image or webcam frame and show the predicted alphabet.

3.5 Advantages of the Proposed Approach

Compares multiple algorithms instead of relying on one model without evaluation.

Uses standard metrics to assess performance beyond accuracy alone.

Can support image-upload prediction and optional webcam input.

Provides a modular workflow that can be tested and improved stage by stage.

Creates a foundation for future work such as word formation or text-to-speech, if separately implemented.

3.6 Limitations and Assumptions

The initial version focuses on alphabet gestures represented in still images. A static-image classifier may not capture motion, facial expressions, grammar, or context that are important in full sign languages. Similar-looking gestures can be confused, and performance may decrease when images differ significantly from the training data. Any webcam capability, real-time speed, and model accuracy must be verified experimentally before being reported as achieved.

CHAPTER 4 — TECHNOLOGY AND TOOLS USED

4.1 Python

Python is suitable for the project because it supports data handling, image processing, machine learning, evaluation, and application development through a broad ecosystem of libraries.

4.2 NumPy and pandas

NumPy supports numerical arrays and mathematical operations. pandas can be used to organize dataset metadata, class counts, and evaluation summaries where tabular data is required.

4.3 Image Processing

Image processing tools are used to load images, resize them to a consistent shape, convert color channels if required, normalize pixel values, and prepare data for model input. OpenCV may be used for webcam capture when included in the implementation.

4.4 scikit-learn

scikit-learn provides implementations of Logistic Regression, Random Forest, data splitting, preprocessing utilities, and classification metrics. It is also useful for generating a confusion matrix and classification report.

4.5 TensorFlow/Keras or PyTorch

A deep learning framework can be used to define, train, validate, and save the CNN. The report should name the framework actually used rather than listing both as if both were implemented.

4.6 Visualization and Development Tools

Matplotlib or a similar visualization library can display class distributions, sample images, training curves, and confusion matrices. Visual Studio Code, Jupyter Notebook, or Google Colab can be used for editing and running code.

CHAPTER 5 — SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

1) Dataset input: labeled images and class labels.

2) Preprocessing: image loading, resizing, normalization, and optional augmentation.

3) Dataset split: training data for fitting and validation/testing data for model assessment.

4) Model training: Logistic Regression, Random Forest, and CNN experiments.

5) Evaluation: metrics and confusion matrix on held-out data.

6) Model selection: choose the model based on evaluation and practical needs.

7) Prediction interface: accept an image or webcam frame, preprocess it, predict the class, and display the alphabet.

5.2 System Workflow

Start → Load dataset → Inspect labels and images → Preprocess images → Split data → Train candidate models → Evaluate each model → Compare metrics → Select and save model → Receive image/webcam frame → Apply the same preprocessing → Predict alphabet → Display result → End.

5.3 Data Flow

The training flow transfers labeled images into preprocessing and then into the training algorithms. The trained models are evaluated using held-out samples. During inference, an input image passes through the same preprocessing steps before it is sent to the saved model. The model returns a class index or class score, which is mapped to the corresponding alphabet label for display.

5.4 Module Description

Module	Responsibility
Dataset loader	Reads images and their class labels from the selected dataset.
Preprocessing module	Resizes images, scales pixel values, and performs any required format conversion.
Augmentation module	Applies suitable transformations to training images only, where appropriate.

Training module	Fits the selected traditional classifiers and/or CNN.
Evaluation module	Calculates accuracy, precision, recall, F1-score, and a confusion matrix.
Model storage module	Saves the trained model and the class-label mapping for later predictions.
Prediction module	Loads an image or frame, preprocesses it, and predicts its class.
User interface	Displays the input and predicted alphabet; it is included only if implemented.

5.5 Input and Output Design

Input: a labeled image during training; an uploaded hand image or webcam frame during prediction, if supported. Output: the predicted alphabet label and, where available, a confidence score. Confidence scores should be presented carefully because they are not necessarily calibrated probabilities. If the image is unclear or outside the trained classes, the application should ideally show an appropriate warning rather than implying certainty.

CHAPTER 6 — METHODOLOGY AND IMPLEMENTATION

6.1 Dataset Preparation

First, identify and download a suitable labeled dataset and record its source. Inspect the number of classes and examples per class. Check whether the dataset labels correspond to the alphabet gestures that the application intends to recognize. Remove unreadable or invalid files using a documented rule. Split the dataset into training and testing subsets, ideally using a stratified split so that class proportions are maintained. If multiple images come from the same person or recording session, consider a split that avoids nearly identical samples appearing in both training and testing data.

6.2 Image Preprocessing

- Resize images to the input dimensions expected by the selected model.
- Convert images to the required color format and numeric type.
- Normalize pixel values consistently, for example by scaling them to a range expected by the model.
- Encode class labels into numeric labels where required.
- Apply augmentation only to training samples, and use transformations that preserve the meaning of a gesture.

6.3 Model Training

For Logistic Regression and Random Forest, image arrays may be flattened into feature vectors or represented by suitable features. Fit any preprocessing transformations on training data only. For the CNN, preserve image height, width, and channel dimensions. Use a validation subset or validation split to monitor training and tune model settings. Save the final selected model together with the class-label order and preprocessing configuration.

6.4 Evaluation Metrics

Accuracy is the proportion of all predictions that are correct. Precision measures how many predictions for a class are correct, recall measures how many true examples of that class are detected, and F1-score combines precision and recall. The provided script reports accuracy, weighted precision, weighted recall, and weighted F1-score for each model. It also saves a CNN confusion matrix as cnn_confusion_matrix.png and a CNN training/validation accuracy plot as cnn_training_accuracy.png. Insert actual results from a real run; no numerical results are assumed in this report.

Accuracy = correct predictions / total predictions.

Precision = TP / (TP + FP); Recall = TP / (TP + FN).

F1-score = 2 × (Precision × Recall) / (Precision + Recall).

A confusion matrix compares actual classes with predicted classes. Its off-diagonal values show which classes are being confused. Metric values should be calculated from the held-out test set; do not invent or estimate results for the report.

6.5 Algorithm / Pseudocode

BEGIN
Load labeled sign-language alphabet dataset
Inspect classes and remove invalid samples using documented rules
Split data into training, validation, and test sets
Preprocess images consistently
Apply augmentation to training data only, if appropriate
Train Logistic Regression baseline
Train Random Forest baseline
Train CNN model
Evaluate each trained model on held-out test data
Compare accuracy, precision, recall, F1-score, and confusion matrix
Select model based on measured performance and practical requirements
Save model and class-label mapping
For each user input image or webcam frame:
Preprocess input using the saved configuration
Predict class with selected model

Map class index to alphabet label
Display predicted label
END

6.6 Prediction Interface

A minimal interface can provide an image-upload control and a prediction button. If webcam support is implemented, the application should request camera permission, capture a frame, apply the same preprocessing used in training, and display the predicted alphabet. A stable real-time implementation should avoid retraining the model for every frame and should load the saved model once when the application starts.

6.7 Reproducibility and Responsible Use

Record dataset source, split proportions, random seed, preprocessing steps, model parameters, library versions, and evaluation results. Do not include private images without permission. Explain that the prototype recognizes only the gesture classes present in its training data and may fail under unfamiliar lighting, backgrounds, hand orientations, or camera conditions.

CHAPTER 7 — TESTING, EVALUATION AND DEMONSTRATION

7.1 Functional Testing

Test	Expected behavior	Actual result
Dataset loading	Dataset files and labels load without errors.	Record pass/fail after testing.
Preprocessing	Input images have the expected dimensions and value range.	Record pass/fail after testing.
Training	Each intended model completes training.	Record model and configuration.
Evaluation	Metrics are generated using held-out test data.	Insert actual metric values.
Image prediction	A supported image produces a predicted class label.	Insert screenshot/result.
Webcam prediction	Camera input produces predictions, if this feature is implemented.	Record only if tested.
Invalid input	Unsupported or unreadable images are handled with a clear message.	Record pass/fail after testing.

7.2 Model Comparison Results

Complete the following table only after running the experiments. The values below are intentionally blank because no measured model results were supplied in the source files.Add the confusion matrix for the selected model and briefly discuss the most frequently confused classes. State the test-set size and whether precision, recall, and F1-score are macro, micro, or weighted averages.

7.3 Sample Input and Output

Example format (illustrative only): Input: an image showing one alphabet gesture. Output: Predicted alphabet = [predicted class]. Replace the bracketed text with a real output from the running application.

7.4 Screenshots

Insert genuine screenshots of:

(1) dataset samples or class distribution,

(2) training or notebook execution,

(3) evaluation metrics and confusion matrix,

(4) the prediction interface. Do not use illustrative screenshots as evidence of completed execution.

7.5 Demonstration Checklist

- Show the dataset source and class labels.
- Explain preprocessing and the train/test split.
- Describe the three algorithms and how they differ.
- Show measured evaluation metrics and the confusion matrix.
- Submit an image and show the predicted alphabet.
- Demonstrate webcam input only if it has been implemented and tested.
- Explain limitations and future improvements.

CHAPTER 8 — CONCLUSION AND FUTURE ENHANCEMENTS

8.1 Summary of the Project

SignVision is designed as a machine learning prototype for recognizing sign language alphabet gestures from images. It organizes the task into dataset preparation, image preprocessing, model training, evaluation, and prediction. The planned comparison of Logistic Regression, Random Forest, and CNN provides a structured way to examine different classification approaches.

8.2 Objectives and Results

The provided scripts implement dataset loading, label encoding, stratified train/test splitting, Logistic Regression and Random Forest training, CNN training with augmentation, classification metrics, a CNN confusion matrix, a CNN training/validation accuracy plot, and webcam prediction using the saved Random Forest model. The final report should state the actual measured results and confirm which features worked during demonstration. No accuracy value or best-model claim is made here because output metrics were not provided.

8.3 Skills and Learning Outcomes

Dataset inspection and preprocessing.

- Supervised learning and image classification.
- Training, validation, and testing practices.
- Classification metrics and confusion-matrix interpretation.
- Model saving and inference pipeline design.
- Awareness of accessibility goals and machine learning limitations.

8.4 Future Enhancements

- Expand and diversify the dataset to include different hands, lighting, and backgrounds.
- Improve robustness using carefully selected augmentation and model tuning.
- Optimize inference speed for webcam use.
- Combine alphabet predictions into words using a separate sequence-building mechanism.
- Explore continuous sign recognition using temporal models and appropriate video datasets.
- Add text-to-speech output, multilingual interface options, and user feedback features.
- Evaluate with users and real-world images while following consent and privacy practices.

CHAPTER 9 — REFERENCES

Replace or supplement these general references with the exact dataset, library versions, and research papers used during implementation.

– Scikit-learn documentation, “Classification metrics” and model documentation. https://scikit-learn.org/stable/

– TensorFlow documentation, tutorials and API reference (if TensorFlow/Keras is used). https://www.tensorflow.org/

– PyTorch documentation (if PyTorch is used). https://pytorch.org/docs/

– OpenCV documentation (if OpenCV is used). https://docs.opencv.org/

– The official source page and license for the sign language alphabet dataset actually used. Add dataset title, author/organization, URL, and access date.

– Any research papers consulted for sign language recognition, image classification, or CNN-based gesture recognition. Add complete bibliographic details.

APPENDIX A — PROJECT DETAILS TO FINALIZE

Dataset name and source URL: _____

Number of alphabet classes: _____

Number of images and split proportions: _____

Python and library versions: _____

SignVision | Page 9 Models actually implemented: _____

Best model based on measured test results: _____

Measured accuracy / precision / recall / F1-score: _____

Webcam input implemented and tested? _____

Application framework / launch command: _____

APPENDIX B — FINAL SUBMISSION CHECKLIST

- Confirm course, team, guide, section, and academic-year details.
- Insert the dataset citation and license.
- Remove any model or feature not actually implemented.
- Fill in actual metrics and include the confusion matrix.
- Insert screenshots from the real application or notebook.
- Update the table of contents and page numbers in Word.
- Proofread names, formatting, references, and declaration.

CHAPTER 10 — IMPLEMENTATION NOTES BASED ON THE PROVIDED CODE

10.1 Training script (main.py)

• Dataset path is currently configured as C:\Users\srini\OneDrive\Desktop\SignVision\dataset\asl_alphabet_train\asl_alphabet_train . Update TRAIN_PATH to match the actual computer.

• At most 300 readable images are loaded per class.

• Traditional models use grayscale 32×32 images normalized to 0–1 and flattened to 1,024 features.

• The CNN uses RGB 64×64 images, normalization, rotation/zoom/translation augmentation, five epochs, and batch size 32.

• Outputs include models/label_encoder.pkl, models/logistic_regression.pkl, models/random_forest.pkl, models/signvision_cnn.keras, cnn_confusion_matrix.png, and cnn_training_accuracy.png.

10.2 Webcam script (websam.py)

• It loads models/random_forest.pkl and models/label_encoder.pkl, so main.py must complete successfully first.

• The camera tries device index 0 and then 1. Close other camera applications if the webcam cannot open.

• The central camera crop is classified. Keep the hand inside the green rectangle and test under several backgrounds and lighting conditions.

• A confidence threshold of 0.60 controls whether a prediction can be added to the sentence; confidence is not a guarantee of correctness.

• ENTER adds a confident predicted letter; SPACE inserts a space; BACKSPACE deletes a character; C clears the sentence; Q exits

10.3 Items to verify before final submission

• Run main.py and enter the actual measured metrics in the results table.

• Add the exact dataset source URL and license to References.

• Verify the configured dataset path and package versions.

• Test the mismatch between full-image training preprocessing and central-crop webcam preprocessing.

• Confirm which class folders exist, including any special labels such as space or del.